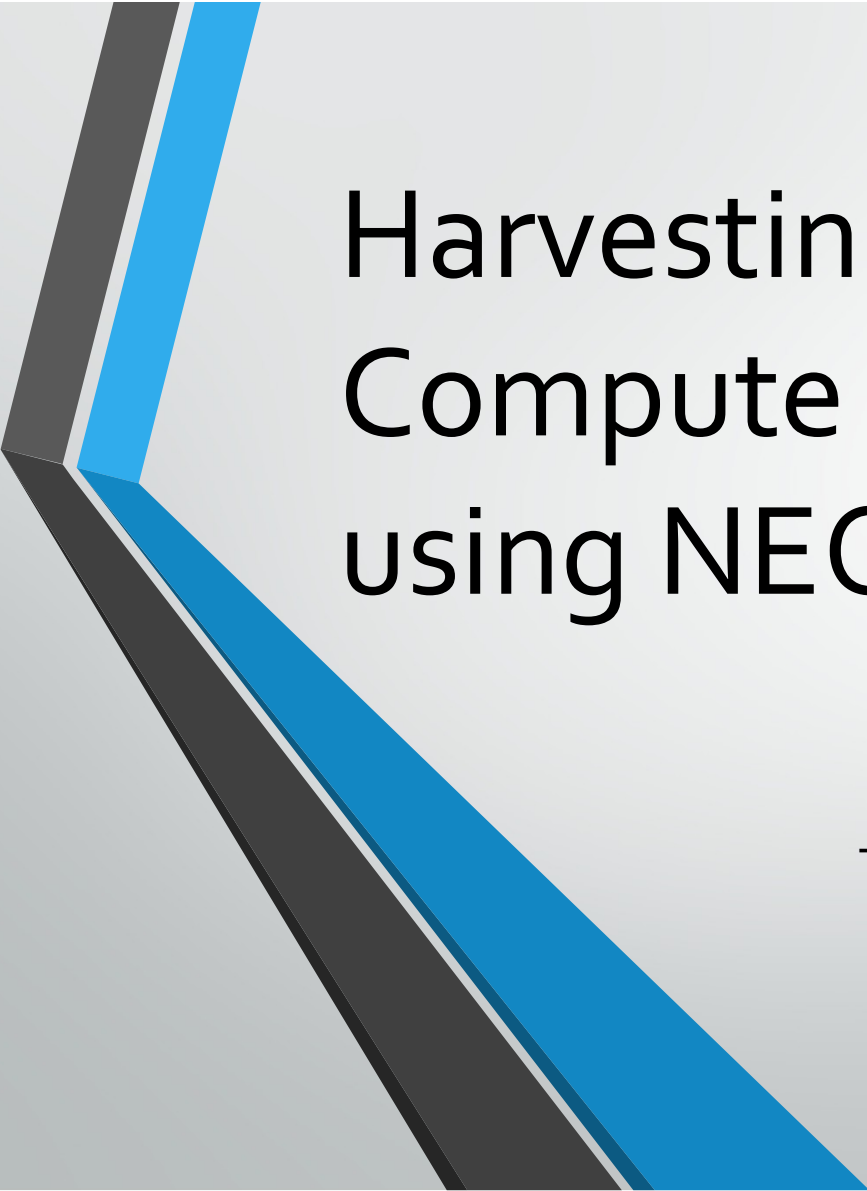




# Harvesting Existing Compute power in ARM using NEON Coprocessor

Mentor: Dr. Sudheendra Kumar

Team members: Hrishikesh, Eshaan, Srikar

# The Neon CoProcessor: ARMv8 with SIMD

- While the ARMv8 architecture has impressive processing power, the **Neon** extension takes it a step further by harnessing the magic of **SIMD (Single Instruction, Multiple Data)**. Think of it as a parallel processing party, where one instruction applies to multiple pieces of data simultaneously, leading to significant performance gains in specific tasks.
- Neon operates on Thirty-Two 128-bit registers, each capable of containing multiple lanes of data; allowing parallel processing on various data types like integers, floats, and bytes. The floating-point and NEON use a common register bank for their operations.
- Its instruction set offers a wide range of operations, from basic arithmetic to complex mathematical functions.
- Applications that can benefit from Neon technology include multimedia and signal processing, 3D graphics, speech, image processing, or other applications where **fixed and floating-point performance** is critical. We will discuss the possibilities and extremes that NEON can achieve.

## Single Instruction Single Data

Each operation specifies the single data source to process. Processing multiple data items requires multiple instructions. The following code uses four instructions to add eight registers:

```
add r0, r5  
add r1, r6  
add r2, r7  
add r3, r8
```

This method is slow and it can be difficult to see how different registers are related. To improve performance and efficiency, media processing is often off-loaded to dedicated processors such as a *Graphics Processing Unit* (GPU) or Media Processing Unit which can process more than one data value with a single instruction.

On an ARM 32-bit processor, performing large numbers of individual 8-bit or 16-bit operations does not use machine resources efficiently because processor, registers, and data path are all designed for 32-bit calculations.

## Single Instruction Multiple Data (vector mode)

An operation can specify that the same processing occurs for multiple data sources. If the LEN value is 4 in the control register, the single vector add instruction performs four adds:

```
VADD.F32 S24, S8, S16
// four operations occur
// S24 = S8 + S16
// S25 = S9 + S17
// S26 = S10 + S18
// S27 = S11 + S20
```

Although there is only one instruction, processing the four additions occurs sequentially in four steps. In ARM terminology this is called *Vector Floating Point*.

*The Vector Floating Point* (VFP) extension was introduced in the ARMv5 architecture and executed short vector instructions to speed up floating point operations. The source and destination registers could be either single registers for scalar operations or a sequence of two to eight registers for vector operations.

*SIMD* operations perform vector calculation more efficiently than VFP operations, vector mode operation was deprecated from the introduction of ARMv7, and replaced with NEON technology which performs multiple operations on wide registers as shown next.

## Single Instruction Multiple Data (packed data mode)

An operation can specify that the same processing occurs for multiple data fields stored in one large register:

```
VADD.I16 Q10, Q8, Q9
```

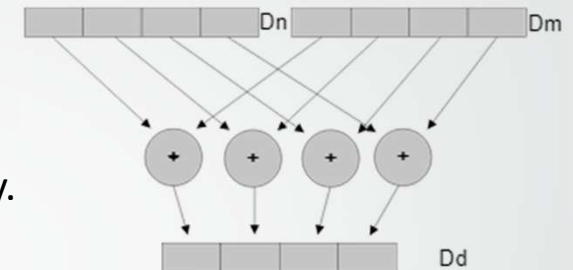
- One operation adds two 64-bit registers.
- Each of the four 16-bit lanes in the register is added separately.
- There are no carries between the lanes .

The single instruction operates on all data values in the large register at the same time. In ARM terminology this is called *Advanced SIMD technology* or *NEON technology*.

Figure shows a 64-bit register holding four 16-bit values, but other combinations are possible for NEON registers:

- Two 32-bit, four 16-bit, or eight 8-bit integer data elements can be operated on simultaneously in a single 64-bit register.
- Four 32-bit, eight 16-bit, or sixteen 8-bit integer data elements can be operated on simultaneously in a single 128-bit register.

Media processors, such as used in mobile devices, often split each full data register into multiple sub-registers and perform computations on the sub-registers in parallel. If the processing for the data sets are simple and repeated many times, SIMD can give considerable performance improvements. It is particularly beneficial for digital signal processing or multimedia algorithms, such as Audio, video, and image processing codecs, 2D graphics based on rectangular blocks of pixels, 3D graphics, Color-space conversion, Physics simulations.



# Vectorization

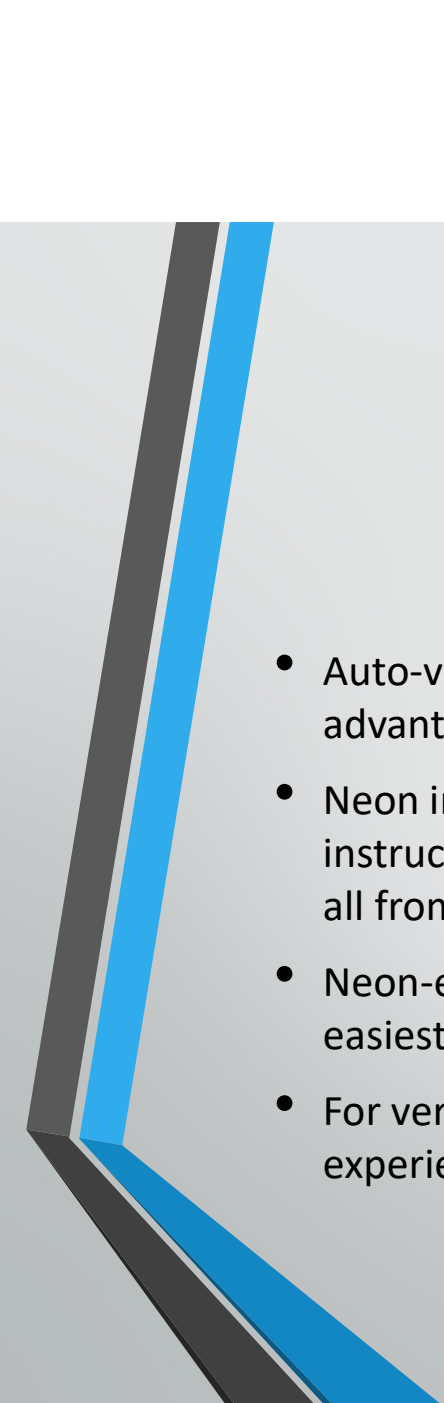
Vectorization is a process by which mathematical operations found in loops in scientific code are executed in parallel on special vector hardware found in CPUs and coprocessors such as NEON.

NEON was designed as an additional load/store architecture to provide good vectorizing compiler support for portable HIGH level languages such as C/C++. This enables a high level of parallelism. You can hand-code NEON instructions for applications that need very high performance. It includes low cost promotion and demotion of data sizes. It also includes structure loads capable of accessing multiple data streams which are interleaved in memory.

NEON instructions can be written as part of the normal ARM code. This makes NEON programming simpler and more efficient than with an external hardware accelerator. There are NEON instructions available to read and write external memory, move data between NEON registers and other ARM registers and to perform SIMD operations.

# Comparison between NEON technology and other SIMD solutions

| NEON technology compared with MMX and AltiVec |                                       |                                                                                       |                                              |
|-----------------------------------------------|---------------------------------------|---------------------------------------------------------------------------------------|----------------------------------------------|
|                                               | NEON technology                       | x86 MMX/SSE                                                                           | AltiVec                                      |
| Number of registers                           | 32 × 128-bit                          | SSE2: 8 × 128-bit XMM<br>(in x86-32 mode)<br>Additional 8 registers<br>in x86-64 mode | 32 × 128-bit                                 |
| Memory / register operations                  | Register-based 3-operand instructions | Mix of register and memory operations                                                 | Register-based 3- and 4-operand instructions |
| Load/store support for packed data types      | Yes as 2,3, or 4 element              | No                                                                                    | No                                           |
| Move between scalar and vector register       | Yes                                   | Yes                                                                                   | No                                           |
| Floating-point support                        | Single-precision 32-bit               | Single-precision and Double-precision                                                 | Single-precision                             |



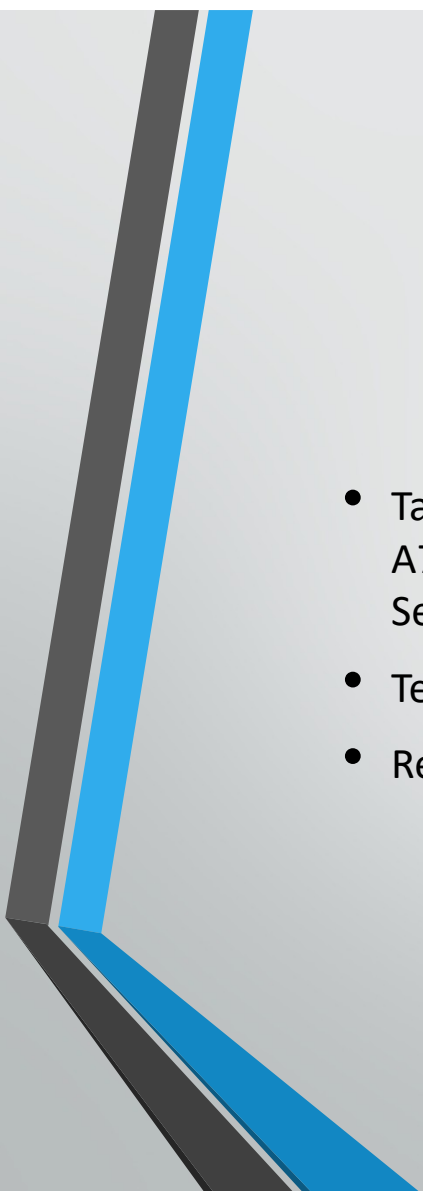
# Methods to Utilize NEON

- Auto-vectorization features in your compiler can automatically optimize your code to take advantage of Neon.
- Neon intrinsics are function calls that the compiler replaces with appropriate Neon instructions. This gives you direct, low-level access to the exact Neon instructions you want, all from C, or C++ code.
- Neon-enabled open source libraries such as the Arm Compute Library provide one of the easiest ways to take advantage of Neon.
- For very high performance, hand-coded Neon assembler can be the best approach for experienced programmers.

# NEON Libraries

There are many other free open source softwares which makes use of NEON, for example:

- Ne10 library functions, the C interfaces to the functions provide assembler and NEON implementations. See <http://projectne10.github.com/Ne10/>.
- OpenMAX, a set of APIs for processing audio, video, and still images. It is part of a standard created by the Khronos group. There is a free ARM implementation of the OpenMAX DL layer for NEON. See <http://www.khronos.org/openmax/>.
- ffmpeg, a collection of codecs for many different audio and video standards under LGPL license at <http://ffmpeg.org/>.
- Eigen3, a linear algebra, matrix math C++ template library at [eigen.tuxfamily.org/](http://eigen.tuxfamily.org/).
- Pixman, a 2D graphics library (part of Cairo graphics) at <http://pixman.org/>.
- x264, a rights-free GPL H.264 video encoder at <http://www.videolan.org/developers/x264.html>.
- Math-neon at <http://code.google.com/p/math-neon/>.

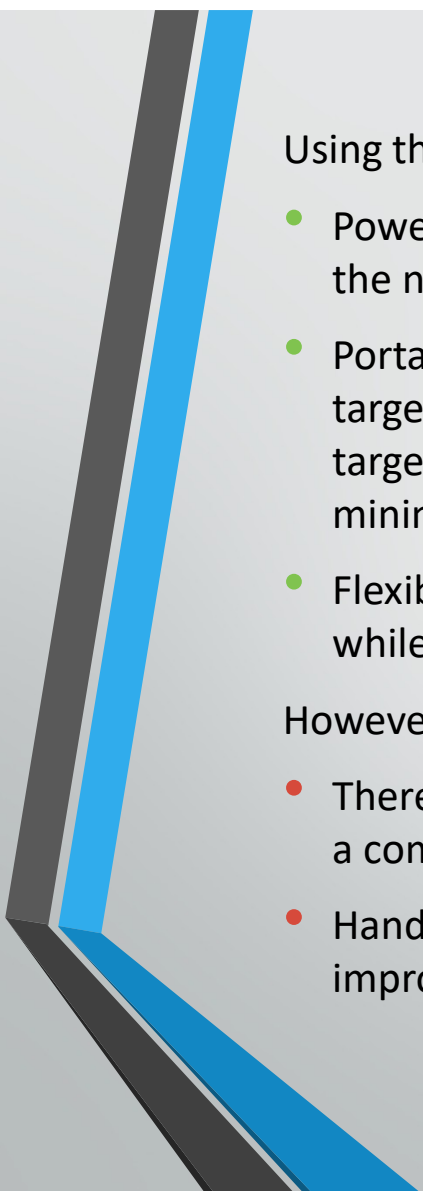


# Project Requirements

- Target Board: Raspberry Pi 4b+ (Broadcom BCM2711, Quad core Cortex-A72 (ARM v8) 64-bit SoC @ 1.8GHz, 2GB LPDDR4-3200 SDRAM) (Ubuntu Server 22.04 LTS)
- Test PC: Windows 11 (Intel i7 12<sup>th</sup> Gen, RTX 3060, 16 GB RAM)
- Remote Connection: Microsoft Visual Studio (SSH Connection)

# Neon intrinsics

- Intrinsics are functions whose precise implementation is known to a compiler. The Neon intrinsics are a set of C and C++ functions defined in `arm_neon.h` which are supported by the Arm compilers and GCC.
- These functions let you use Neon without having to write assembly code directly, since the functions themselves contain short assembly kernels which are inlined into the calling code. Additionally, register allocation and pipeline optimization are handled by the compiler so many difficulties faced by the assembly programmer are avoided.



Using the Neon intrinsics has a number of benefits:

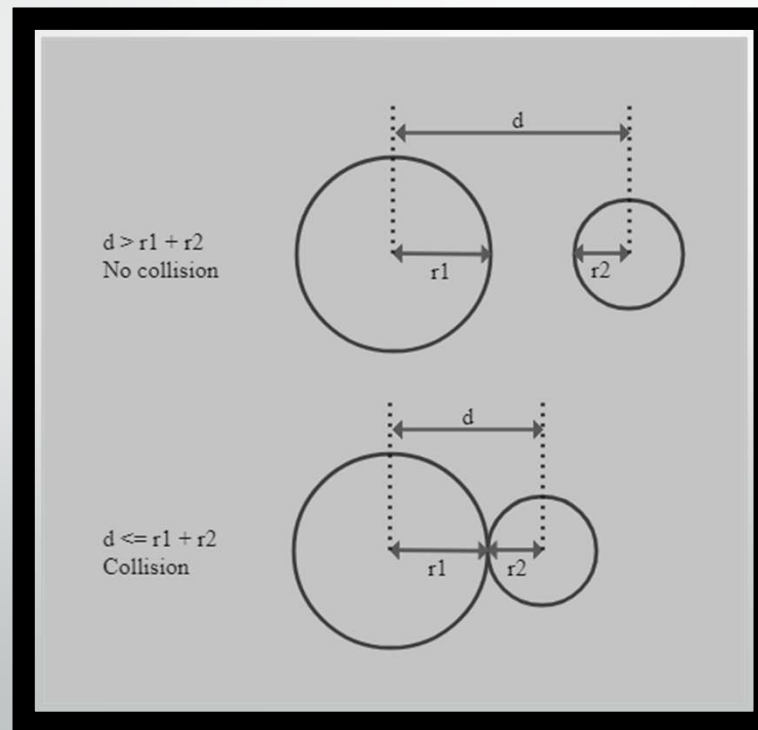
- Powerful: Intrinsics give the programmer direct access to the Neon instruction set without the need for hand-written assembly code.
- Portable: Hand-written Neon assembly instructions might need to be rewritten for different target processors. C and C++ code containing Neon intrinsics can be compiled for a new target or a new execution state (for example, migrating from AArch32 to AArch64) with minimal or no code changes.
- Flexible: The programmer can exploit Neon when needed or use C/C++ when it isn't needed, while avoiding many low-level engineering concerns.

However, intrinsics might not be the right choice in all situations:

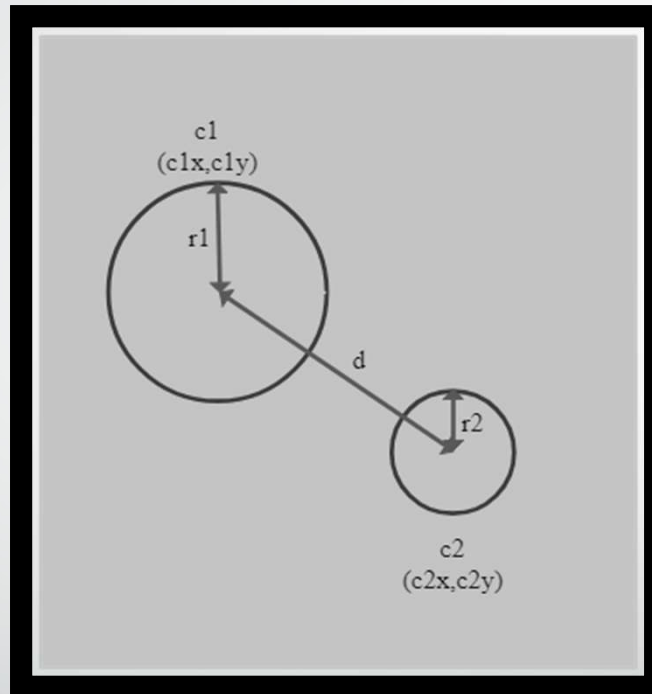
- There is a steeper learning curve to use Neon intrinsics than importing a library or relying on a compiler.
- Hand-optimized assembly code might offer the greatest scope for performance improvement even if it is more difficult to write.

# Example : Collision Detection

- To decide whether two objects have collided or not, we can simplify their shape to circles.
- If we first consider collision detection along a single axis, we can see that collisions occur when these circles overlap.



In two dimensions, each object has a pair of (x,y) coordinates specifying the center of the circle. Consider the same two circles with centre point's c1 and c2, at positions (c1x,c1y) and (c2x,c2y) respectively.



Using distance formula for 2 dimensions we can write:

$$d^2 = (c1y - c2y)^2 + (c1x - c2x)^2$$

# Algorithm implementation without vectorization

```
#include <stdio.h>
#include <stdbool.h>

struct circle {
    float radius;
    float x;
    float y;
};

bool does_collide(struct circle c1, struct circle c2) {
    // Two circles collide if the distance from c1 to c2 is less
    // than the sum of their radii, or equivalently if the squared
    // distance is less than the square of the sum of the radii.
    float delta_x = c1.x - c2.x;
    float delta_y = c1.y - c2.y;
    float deltas_squared = (delta_x * delta_x) + (delta_y * delta_y);
    float radius_sum_squared = (c1.radius * c1.radius) + (c2.radius * c2.radius);
    return deltas_squared <= radius_sum_squared;
}

int main() {
    struct circle c1;
    c1.radius = 2.0;
    c1.x = 2.0;
    c1.y = 4.0;

    struct circle c2;
    c2.radius = 1.0;
    c2.x = 6.0;
    c2.y = 1.0;

    if (does_collide(c1, c2)) {
        printf("Circles collide\n");
    }
    else {
        printf("Circles do not collide\n");
    }
    return 0;
}
```

# How to execute?

- Flash Raspberry Pi's boot SD Card with Ubuntu Server 22.04 LTS OS make sure configure prior itself. This can be easily performed with the help of software's such RPI Imager ( Tutorial - [https://youtu.be/Cw\\_34fuve6E?si=9VdskgPn1mZil2sU](https://youtu.be/Cw_34fuve6E?si=9VdskgPn1mZil2sU) ) Make sure to note down Pi's IP Address.
- Install VS Code on Test PC and install Remote SSH extension. Connect to Pi with the IP Address noted from previous step ( Tutorial - <https://youtu.be/BkQ2hL0fkWM?si=u4k4LswNMKU3PDSI> ).
- Install gcc and g++ which are C and C++ compilers respectively ( Tutorial - <https://youtu.be/9InvWdigrks?si=u5alBOKkwOm81GmF> ).
- To run c codes, create a file with <name>.c extension and enter the code. After saving the file run the following commands in the terminal :

```
pi@ubuntu:~ gcc <name>.c  
pi@ubuntu:~ ./a.out
```

# Basic vectorization using Neon intrinsics

```
#include <arm_neon.h>
#include <stdio.h>
#include <stdbool.h>

struct circle {
    float y;
    float x;
    float radius;
} __attribute__((aligned (64)));

bool does_collide_neon(struct circle const c1, struct circle const c2) {
    float32x2_t c1_coords = vld1_f32(&c1.y);
    float32x2_t c2_coords = vld1_f32(&c2.y);
    float32x2_t deltas = vsub_f32(c1_coords, c2_coords);
    float32x2_t deltas_squared = vmul_f32(deltas, deltas);
    float sum_deltas_squared = vpaddsf32(deltas_squared);
    float radius_sum = c1.radius + c2.radius;
    float radius_sum_squared = radius_sum * radius_sum;
    return sum_deltas_squared <= radius_sum_squared;
}

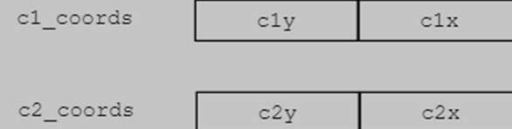
int main() {
    struct circle c1;
    c1.radius = 2.0;
    c1.x = 2.0;
    c1.y = 4.0;

    struct circle c2;
    c2.radius = 1.0;
    c2.x = 6.0;
    c2.y = 1.0;

    if (does_collide_neon(c1, c2)) {
        printf("Circles collide\n");
    } else {
        printf("Circles do not collide\n");
    }
    return (0);
}
```

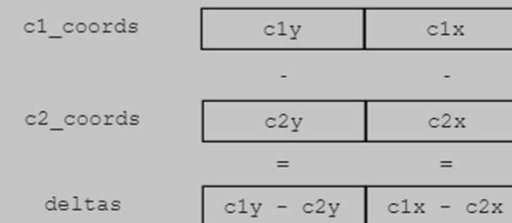
This code uses Neon intrinsics with 2-lane 64-bit Neon registers, each containing 32-bit floating-point values to parallelize parts of the algorithm.

```
float32x2_t c1_coords = vld1_f32(&c1.y);  
float32x2_t c2_coords = vld1_f32(&c2.y);
```



The `vld1_f32` intrinsic loads two 32-bit floating-point values into the Neon register, as the following diagram shows:

```
float32x2_t deltas = vsub_f32(c1_coords, c2_coords);
```



The `vsub_f32` intrinsic subtracts the vector elements in the second Neon register from the corresponding elements in the vector in the first register, and then places each result into elements of a destination register.

```
float32x2_t deltas_squared = vmul_f32(deltas, deltas);
```

|                |                 |                 |
|----------------|-----------------|-----------------|
| deltas         | $c1y - c2y$     | $c1x - c2x$     |
|                | x               | x               |
| deltas         | $c1y - c2y$     | $c1x - c2x$     |
|                | =               | =               |
| deltas_squared | $(c1y - c2y)^2$ | $(c1x - c2x)^2$ |

The `vmul_f32` multiplies the 32-bit floating-point vector elements in the first Neon register by the values in the second source, and places the results in a vector. In our example, we use the same values to obtain the square of the deltas.

```
float sum_deltas_squared = vpaddd_f32(deltas_squared);
```

The `vpadd_f32` adds the two floating-point vector elements in the Neon register to give a single scalar result.

This code vectorizes the implementation of the algorithm by parallelizing the subtraction and multiplication operations when computing the square of the distance.

# Advanced vectorization using Neon intrinsics

The solution presented in Basic vectorization using Neon intrinsics may not be faster than the original un-vectorized implementation.

The performance gain from parallelizing the subtraction and multiplication operations is limited by the memory layout of our input. Loading the vector registers from this memory layout requires several instructions. Also, we only parallelized two operations, a subtraction and a multiplication, before needing to perform a cross-lane addition operation.

The declaration of the structure `circle` means that the data is interleaved, which inhibits vectorization.

An alternative approach is to change our data layout to provide deinterleaved data.

The following example shows this approach. It creates a new structure `circles` to hold pointers to arrays containing deinterleaved `x`, `y`, and `radius` data. This approach lets us check for collisions between a single collider circle and multiple input circles.

```

#include <arm_neon.h>
#include <stdio.h>
#include <stdbool.h>
// Structure containing data for a single collider circle
struct circle {
    float x;
    float y;
    float radius;
};
// Structure containing an array of pointers to data for multiple circles
struct circles {
    size_t size;
    float* xs;
    float* ys;
    float* radii;
};
void does_collide_neon_deinterleaved(struct circles const input, struct circle collider, bool* out) {
    // Duplicate the collider properties in 3 separate 4-lane vector registers
    float32x4_t c1_x = vdupq_n_f32(collider.x);
    float32x4_t c1_y = vdupq_n_f32(collider.y);
    float32x4_t c1_r = vdupq_n_f32(collider.radius);

    for (size_t offset = 0; offset != input.size; offset += 4) {
        // Perform 4 collision tests at a time
        float32x4_t x = vld1q_f32(input.xs + offset);
        float32x4_t y = vld1q_f32(input.ys + offset);

        float32x4_t delta_x = vsubq_f32(c1_x, x);
        float32x4_t delta_y = vsubq_f32(c1_y, y);
        float32x4_t delta_x_squared = vmulq_f32(delta_x, delta_x);
        float32x4_t delta_y_squared = vmulq_f32(delta_y, delta_y);
        float32x4_t sum_deltas_squared = vaddq_f32(delta_x_squared, delta_y_squared);

        float32x4_t r = vld1q_f32(input.radii + offset);
        float32x4_t radius_sum = vaddq_f32(c1_r, r);
        float32x4_t radius_sum_squared = vmulq_f32(radius_sum, radius_sum);
        uint32x4_t mask = vcltq_f32(sum_deltas_squared, radius_sum_squared);

        // Unpack the results in each lane
        out[offset] = 1 & vgetq_lane_u32(mask, 0);
        out[offset + 1] = 1 & vgetq_lane_u32(mask, 1);
        out[offset + 2] = 1 & vgetq_lane_u32(mask, 2);
        out[offset + 3] = 1 & vgetq_lane_u32(mask, 3);
    }
}

```

```

int main() {
    int num_input = 4;
    float input_x[num_input] __attribute__((aligned (64)));
    float input_y[num_input] __attribute__((aligned (64)));
    float input_r[num_input] __attribute__((aligned (64)));
    bool output[num_input] __attribute__((aligned (64)));
    // Set up the data for multiple circles
    for (int i = 0; i < num_input; i++) {
        input_x[i] = i*2;
        input_y[i] = i*3;
        input_r[i] = i;
        output[i] = 0;
    }
    // Create input object containing pointers to array data for multiple circles
    struct circles c1;
    c1.size = num_input;
    c1.radii = input_r;
    c1.xs = input_x;
    c1.ys = input_y;
    // Create collider object containing data for a single circle
    struct circle c2;
    c2.radius = 5.0;
    c2.x = 10.0;
    c2.y = 10.0;
    // Test whether the collider circle collides with any of the input circles, returning results in output
    does_collide_neon_deinterleaved(c1, c2, output);
    // Iterate over the returned output data and display results
    for (int i = 0; i < num_input; i++) {
        if (output[i]) {
            printf("Circle %d at (%.1f, %.1f) with radius %.1f collides\n", i, input_x[i], input_y[i], input_r[i]);
        } else {
            printf("Circle %d at (%.1f, %.1f) with radius %.1f does not collide\n", i, input_x[i], input_y[i], input_r[i]);
        }
    }
    return 0;
}

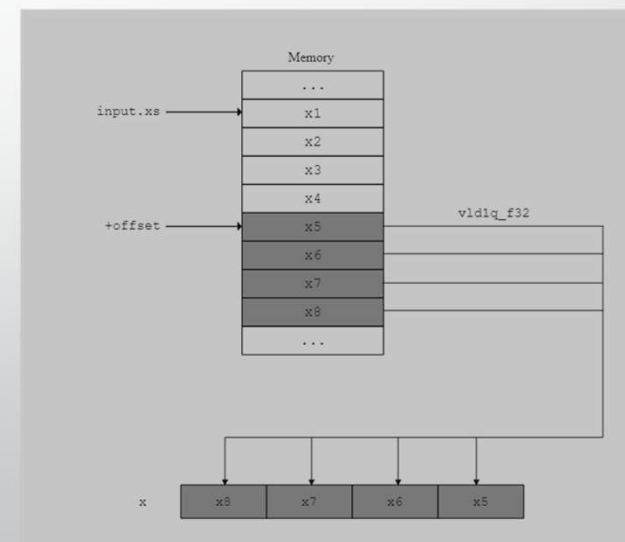
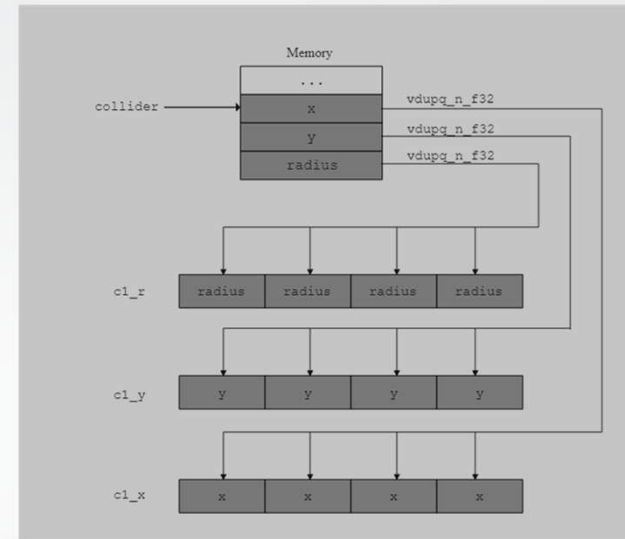
```

```
float32x4_t c1_x = vdupq_n_f32(collider.x);
float32x4_t c1_y = vdupq_n_f32(collider.y);
float32x4_t c1_r = vdupq_n_f32(collider.radius);
```

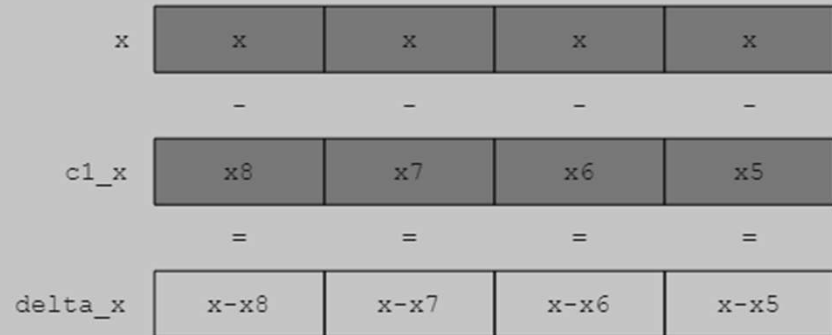
The `vdupq_n_f32` intrinsic duplicates a single scalar value into all the lanes of a Neon register.

```
float32x4_t x = vld1q_f32(input.xs + offset);
float32x4_t y = vld1q_f32(input.ys + offset);
```

The `vld1q_f32` intrinsic loads four 32-bit values from consecutive memory addresses into four 32-bit lanes of a Neon register. The address is calculated by adding an offset to the base address pointer for both the x and y data. The offset increments by four on each loop iteration, because each loop deals with four circles.



```
float32x4_t delta_x = vsubq_f32(c1_x, x);
float32x4_t delta_y = vsubq_f32(c1_y, y);
```



Subtract the collider *c1\_x* value from each of the four input *x* values to obtain the *delta\_x* value, and similar for *delta\_y*.

```
float32x4_t delta_x_squared = vmulq_f32(delta_x, delta_x);
float32x4_t delta_y_squared = vmulq_f32(delta_y, delta_y);
```

Multiply the *delta\_x* and *delta\_y* values with themselves to obtain the squares.

```
float32x4_t sum_deltas_squared = vaddq_f32(delta_x_squared, delta_y_squared);
```

Sum the lanes in the *delta\_x\_squared* and *delta\_y\_squared* separately to obtain the square of the distance.

```
float32x4_t r = vld1q_f32(input.radii + offset);  
float32x4_t radius_sum = vaddq_f32(c1_r, r);  
float32x4_t radius_sum_squared = vmulq_f32(radius_sum, radius_sum);
```

Use a similar process to calculate the squares of the radii.

```
uint32x4_t mask = vcltq_f32(sum_deltas_squared, radius_sum_squared);
```

The `vcltq_f32` intrinsic compares each lane of the first value with the corresponding lane in the second value. If the first value is less than the second, the intrinsic sets every bit of the corresponding lane in the result to 1, or 0 otherwise.

```
out[offset] = 1 & vgetq_lane_u32(mask, 0);  
out[offset + 1] = 1 & vgetq_lane_u32(mask, 1);  
out[offset + 2] = 1 & vgetq_lane_u32(mask, 2);  
out[offset + 3] = 1 & vgetq_lane_u32(mask, 3);
```

Extract each lane from the result individually, and set the next value in the out array to either 1 or 0.



# Performance results

# Program conventions

## Detecting Presence of NEON Unit: Macros

It is a piece of code in a program, similar to a variable which returns a value during pre-processing. Hence, it doesn't consume program space. In order to use the intrinsics the Advanced SIMD architecture must be supported, and some specific instructions may or may not be enabled. When the following macros are defined and equal to 1, the corresponding features are available.

- `__ARM_NEON`: Advanced SIMD is supported by the compiler. Always 1 for AArch64.
- `__ARM_NEON_FP`: Neon floating-point operations are supported. Always 1 for AArch64.
- `__ARM_FEATURE_CRYPTO`: Crypto instructions are available. Cryptographic Neon intrinsics are therefore available.
- `__ARM_FEATURE_FMA`: The fused multiply-accumulate instructions are available. Neon intrinsics which use these are therefore available.

This list is not exhaustive and further macros are detailed in the [Arm C Language Extensions](#) document.

## Data Types

There are three major categories of data type available in `arm_neon.h` which follow these patterns:

- `baseW_t` scalar data types
- `baseWxL_t` vector data types
- `baseWxLxN_t` vector array data types

Where:

- `base` refers to the fundamental data type.
- `W` is the width of the fundamental type.
- `L` is the number of scalar data type instances in a vector data type, for example an array of scalars.
- `N` is the number of vector data type instances in a vector array type, for example a struct of arrays of scalars.

Generally `W` and `L` are such that the vector data types are 64 or 128 bits long, and so fit completely into a Neon register. `N` corresponds with those instructions which operate on multiple registers at once.

## Functions

General Syntax : `ret v[p][q][r]name[u][n][q][x][_high][_lane | laneq][_n][_result]_type(args)`

- `ret` : return type of the function.
- `v` : short for vector.
- `p` : indicates a pairwise operation.
- `q` : indicates a saturating operation.
- `r` : indicates a rounding operation.
- `name` : the descriptive name of the basic operation.
- `u` : indicates signed-to-unsigned saturation.
- `n` : indicates a narrowing operation.
- `q` : postfixing the name indicates an operation on 128-bit vectors.
- `x` : indicates an Advanced SIMD scalar operation in AArch64. It can be one of b, h, s or d (i.e, 8, 16, 32, or 64 bits).
- `_high` : In AArch64, used for widening and narrowing operations involving 128-bit operands(For widening 128-bit operands, high refers to the top 64-bits of the source operand(s). For narrowing, it refers to the top 64-bits of the destination operand).
- `_n` : indicates a scalar operand supplied as an argument.
- `_lane` : indicates a scalar operand taken from the lane of a vector. `_laneq` indicates a scalar operand taken from the lane of an input vector of 128-bit width.
- `type` : the primary operand type in short form.
- `args` : the function's arguments.

# Emulating Neon Intrinsics on Windows

Remotely connecting to Pi is a time consuming and tedious process just to test a few lines of code. An easier alternative is to port neon intrinsics to x86 SIMD. Intel SSE is a SIMD which is similar to NEON. This way, we can test neon intrinsic arm code on a Intel core based windows or any other OS based laptop's.

A [tutorial](#) hosted by intel and [GitHub repository](#) are a great way to understand this process in detail. In this course, we will constrain ourselves only to the implementation for this following topic :

- To run neon intrinsic code on x86, make sure that gcc / g++ are installed and updated to the latest version.
- Replace `#include <arm_neon.h>` with `#include "NEON_2_SSE.h"` and make sure to include NEON\_2\_SSE.h file in program directory. The header file can be easily downloaded from [GitHub repository](#) .

# Auto Vectorization

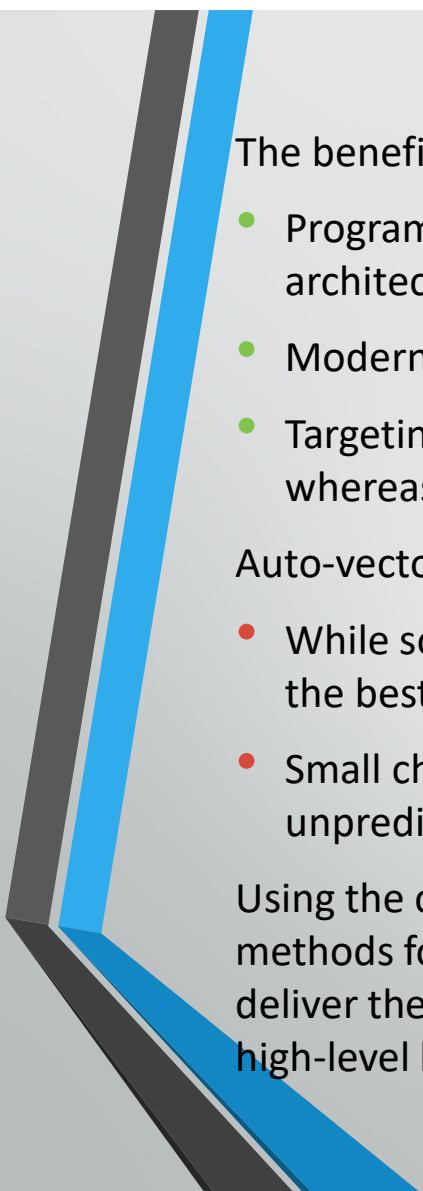
Writing hand-optimized assembly kernels or C code containing Neon intrinsics provides a high level of control over the Neon code in your software. However, these methods can result in significant portability and engineering complexity costs.

In many cases a high quality compiler can generate code which is just as good, but requires significantly less design time. The process of allowing the compiler to automatically identify opportunities in your code to use Advanced SIMD instructions is called auto-vectorization.

In terms of specific compilation techniques, auto-vectorization includes:

- Loop vectorization: unrolling loops to reduce the number of iterations, while performing more operations in each iteration.
- Superword-Level Parallelism (SLP) vectorization: bundling scalar operations together to make use of full width Advanced SIMD instructions.

Auto-vectorizing compilers include [Arm Compiler 6](#), [Arm C/C++ Compiler](#), [LLVM-clang](#), and [GCC](#).



The benefits of relying on compiler auto-vectorization include the following:

- Programs implemented in high level languages are portable, so long as there are no architecture specific code elements such as inline assembly or intrinsics.
- Modern compilers are capable of performing advanced optimizations automatically.
- Targeting a given micro-architecture can be as easy as setting a single compiler option, whereas optimizing an assembly program requires deep knowledge of the target hardware.

Auto-vectorization might not be the right choice in all situations, however:

- While source code can be architecture agnostic, it may have to be compiler specific to get the best code-generation.
- Small changes in a high-level language or the compiler options can result in significant and unpredictable changes in generated code.

Using the compiler to generate Neon code will be appropriate for most projects. Other methods for exploiting Neon only become necessary when the generated code does not deliver the necessary performance, or when particular hardware features are not supported by high-level languages.



# Optimizing for vectorization

The C and C++ languages do not provide syntax that specifies concurrent behavior, so compilers cannot safely generate parallel code. However, the developer can provide additional information to let the compiler know where it is safe to vectorize.

Unlike intrinsics, these modifications are not architecture dependent, and are likely to improve vectorization on any target platform. These modifications usually do not have negative impact on performance on targets where vectorization is not possible.

- **Indicate knowledge of number of loop iterations:** If a loop has a fixed iteration count, or if you know that the iteration count is always a power of 2, making this obvious to the compiler enables the compiler to perform optimizations that would otherwise be unsafe or difficult.
- **Avoid loop-carried dependencies:** If your code contains a loop where the result of one iteration is affected by the result of a previous iteration, the compiler cannot vectorize it. If possible, restructure the code to remove any loop-carried dependencies.
- **Use the restrict keyword:** C99 introduced the restrict keyword, that you can use to inform the compiler that the location accessed through a specific pointer is not accessed through any other pointer within the current scope.
- **Avoid conditions inside loops:** Normally, the compiler cannot vectorize loops containing conditional statements. In the best case, it duplicates the loop, but in many cases it cannot vectorize it at all.

- **Use suitable data types:** For best performance, always use the smallest data type that can hold the required values. This means a NEON register can hold more data elements and execute more operations in parallel. In a given period, the NEON unit can process twice as many 8-bit values as 16-bit values. NEON technology supports 64-bit integers only for certain operations, so avoid using `long long` variables where possible.
- **Floating-point vectorization:** NEON does not support double-precision floating-point, so using a double-precision double where a single-precision float is sufficient can prevent the compiler from vectorizing code. On the contrary, changing the order of the operations or inputs can result in further loss in precision. Hence some floating-point operations are not vectorized by default because vectorizing can change the order of the operations. If the algorithm does not require this level of precision, you can specify `--fpmode=fast`, for `armcc`, or `-ffast-math`, for `GCC`, on the command line to enable these optimizations.
- **IEEE 754 Compliance:** The NEON unit always operates in Flush-to-Zero mode (see Flush-to-zero mode), making it non-compliant with IEEE 754. By default, `armcc` uses `--fpmode=std`, permitting the deviations from the standard. However, if the command line parameters specify a mode option requiring IEEE 754 compliance, for example `--fpmode=ieee_full`, most floating-point operations cannot be vectorized.
- Functions called inside a loop should be [inline](#).
- Avoid using a `break` statement to exit a loop.

# References

- Arm documentation about Neon :-

<https://developer.arm.com/documentation/102467/latest/>

<https://developer.arm.com/documentation/den0018/latest>

- Intrinsics reference :-

<https://developer.arm.com/documentation/den0018/a/NEON-Intrinsics-Reference>

[https://developer.arm.com/architectures/instruction-sets/intrinsics/#f:@navigationhierarchiessimdisa=\[Neon\]](https://developer.arm.com/architectures/instruction-sets/intrinsics/#f:@navigationhierarchiessimdisa=[Neon])



# Questions

We'd love to hear your suggestion and expectations from us!!



Thank You